FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

INTRODUCCIÓN A SISTEMAS DISTRIBUIDOS

Profesor: John Corredor, Ph.D.

TALLER RMI/gRPC

Sistema de Gestión de Biblioteca con gRPC

Presentado por:

Nicolas Leon

Juan Diego Ariza Lopez

Bogotá D.C., Colombia
Noviembre de 2025

Índice

1. Introducción **2**

2. Arquitectura del Sistema **2**
2.1. Contrato del Servicio (biblioteca.proto) 2
2.2. Servidor (GC - Gestor Central) 2
2.3. Cliente (PS - Proceso Solicitante) 2

3. Servicios Remotos **2**
3.1. Consulta por ISBN 3
3.2. Préstamo por ISBN 3
3.3. Préstamo por Título 3
3.4. Devolución 3

4. Implementación **3**
4.1. Contrato gRPC (biblioteca.proto) 3
4.2. Estructura de datos 3
4.3. Implementación del servicio Préstamo por ISBN 4

5. Base de Datos **4**

6. Compilación y Ejecución **4**
6.1. Requisitos 4
6.2. Instalación de dependencias 5
6.3. Compilación 5
6.4. Ejecución en dos computadoras 5

7. Pruebas **5**
7.1. Prueba con un cliente 5
7.2. Prueba con dos clientes simultáneos 6

8. Conclusiones **6**

1. Introducción

El taller tiene como objetivo implementar un sistema de gestión de biblioteca utilizando la tecnología **gRPC** (Google Remote Procedure Call) en C++. gRPC es un framework de comunicación remota de alto rendimiento que permite a clientes y servidores comunicarse de forma transparente a través de la red, como si los métodos remotos fueran llamadas locales.

El sistema implementado sigue una arquitectura cliente-servidor donde el servidor expone cuatro servicios remotos para gestionar el préstamo, devolución y consulta de libros almacenados en un archivo de texto plano que actúa como base de datos.

2. Arquitectura del Sistema

El sistema está compuesto por tres componentes principales:

2.1. Contrato del Servicio (biblioteca.proto)

El archivo `biblioteca.proto` define la interfaz del servicio remoto usando Protocol Buffers. En él se declaran los 4 métodos RPC disponibles y los mensajes que viajan entre cliente y servidor.

2.2. Servidor (GC - Gestor Central)

El servidor implementa los 4 servicios remotos y mantiene el inventario de libros en memoria, persistiendo los cambios en `libros.txt` tras cada operación que modifique el estado.

2.3. Cliente (PS - Proceso Solicitante)

El cliente presenta un menú interactivo al usuario y traduce cada acción en una llamada remota al servidor mediante el stub generado por gRPC.

```
Cliente / PS (PC 2)                Servidor / GC (PC 1)
+------------------+   red/gRPC   +--------------------+
|   cliente.cpp    | -----------> |    servidor.cpp    |
|                  | <----------- |                    |
+------------------+              |    libros.txt      |
                                  +--------------------+
```

Figura 1: Arquitectura cliente-servidor del sistema

3. Servicios Remotos

Los servicios remotos están definidos en `biblioteca.proto` y su implementación se encuentra en `servidor.cpp`.

3.1. Consulta por ISBN

Dado el ISBN de un libro, retorna si el libro existe en la biblioteca y cuántos ejemplares están disponibles. No modifica el inventario.

3.2. Préstamo por ISBN

Dado el ISBN de un libro, verifica si hay ejemplares disponibles. Si los hay, incrementa la cantidad prestada, persiste el cambio y retorna la fecha de devolución (fecha actual + 7 días). Si no hay disponibles, retorna rechazo.

3.3. Préstamo por Título

Funcionalidad equivalente al préstamo por ISBN, pero la búsqueda se realiza por el título exacto del libro.

3.4. Devolución

Dado el ISBN, registra la devolución del libro decrementando la cantidad prestada y persistiendo el cambio en `libros.txt`.

4. Implementación

4.1. Contrato gRPC (biblioteca.proto)

Listing 1: Definición del servicio en biblioteca.proto

```
service GestionBiblioteca {
  rpc PrestamoPorIsbn  (PeticionIsbn)   returns (RespuestaPrestamo);
  rpc PrestamoPorTitulo(PeticionTitulo) returns (RespuestaPrestamo);
  rpc Consulta         (PeticionIsbn)   returns (RespuestaConsulta);
  rpc Devolucion       (PeticionIsbn)   returns (RespuestaDevolucion);
}
```

4.2. Estructura de datos

El inventario se mantiene en memoria como un `unordered_map` indexado por ISBN, lo que permite búsquedas en tiempo constante O(1).

Listing 2: Estructura Libro y mapa de inventario

```
struct Libro {
    std::string isbn;
    std::string titulo;
    int cantidad_total;
    int cantidad_prestada;
};

std::unordered_map<std::string, Libro> inventario;
```

4.3. Implementación del servicio Préstamo por ISBN

Listing 3: Método PrestamoPorIsbn en servidor.cpp

```
Status PrestamoPorIsbn(ServerContext* context,
                       const PeticionIsbn* request,
                       RespuestaPrestamo* reply) override {
    std::string isbn = request->isbn();
    auto it = inventario.find(isbn);

    if (it != inventario.end() &&
       (it->second.cantidad_total - it->second.cantidad_prestada) > 0) {
        it->second.cantidad_prestada++;
        guardarBaseDeDatos();
        reply->set_aprobado(true);
        reply->set_fecha_devolucion(calcularFechaDevolucion());
    } else {
        reply->set_aprobado(false);
    }
    return Status::OK;
}
```

5. Base de Datos

La persistencia se implementa mediante el archivo `libros.txt`, con el siguiente formato por línea:

`isbn,titulo,cantidad_total,cantidad_prestada`

ISBN	Título	Total	Prestados
9786071511471	Sistemas Operativos	5	0
9788448197503	Estructuras de Datos	3	1
9786073236973	Sistemas Distribuidos	4	0
1111111111111	El Hobbit	2	2
9780132350884	Clean Code	3	0
9780201633610	Design Patterns	2	0
9780262033848	Introduction to Algorithms	5	1

Cuadro 1: Inventario inicial de libros

6. Compilación y Ejecución

6.1. Requisitos

- Ubuntu 22.04 o superior
- CMake 3.15+

- g++ con soporte C++17
- Librerías: `libgrpc++-dev`, `libprotobuf-dev`

6.2. Instalación de dependencias

```
sudo apt install -y cmake build-essential \
    libgrpc++-dev libprotobuf-dev protobuf-compiler-grpc
```

6.3. Compilación

```
mkdir build && cd build
cmake ..
make
```

6.4. Ejecución en dos computadoras

En la PC del servidor:

```
cd Taller_RMI_gRPC
./build/servidor
```

En la PC del cliente (con la IP del servidor configurada en `cliente.cpp`):

```
./build/cliente
```

7. Pruebas

7.1. Prueba con un cliente

Se verificó el correcto funcionamiento de los 4 métodos remotos ejecutando el cliente en una máquina y el servidor en otra:

- **Consulta:** Se consultó el ISBN `9786071511471` y el servidor retornó 5 ejemplares disponibles.
- **Préstamo por ISBN:** Se solicitó el mismo libro y el servidor aprobó el préstamo con fecha de devolución a 7 días.
- **Préstamo por Título:** Se solicitó *Sistemas Distribuidos* por título y fue aprobado correctamente.
- **Devolución:** Se devolvió el libro prestado y el servidor actualizó el inventario correctamente.

7.2. Prueba con dos clientes simultáneos

Se ejecutaron dos instancias del cliente desde diferentes terminales conectadas al mismo servidor, verificando que las operaciones de ambos clientes se reflejaban correctamente en el inventario compartido.

8. Conclusiones

- gRPC facilita la comunicación entre procesos remotos al abstraer completamente el manejo de la red, permitiendo que los métodos remotos se llamen como si fueran locales.
- El uso de Protocol Buffers como mecanismo de serialización garantiza una comunicación eficiente y tipada entre cliente y servidor.
- La arquitectura cliente-servidor implementada permite que múltiples clientes accedan concurrentemente al mismo servidor sin conflictos en el inventario.
- El modelo síncrono de gRPC simplifica el flujo de control en el cliente, ya que cada llamada remota espera su respuesta antes de continuar.